Trabalho Prático: Sudoku Genérico como CSP

Objetivo

O objetivo deste trabalho foi fazer um gerador e solver de sudoku n^2 x n^2.

Usei o CP-SAT do OR-Tools porque foi o recomendado.

R1- Grupo genérico de células

Box é uma classe que guarda n para verificar se as coordenadas são válidas e um dicionário (linhas, coluna) → valor ou None se for uma celula livre.

O método add(self, i, j, val=None) adiciona uma célula ao dicionário com o valor padrão None

O método matriz(self) devolve o grupo como uma lista de listas onde as células vazias estão representadas com o valor 0.

In [2]:
```python
from ortools.sat.python import cp_model
from random import randint

class box:
    def __init__(self, n):
        self.n = n
        self.celulas = {}

    def add(self, i, j, val=None):
        N = self.n * self.n

        if i<0 or i≥N or j<0 or j≥N:
            raise ValueError("Coordenadas fora da gralha")

        if val is not None and (val<1 or val>N):
            raise ValueError(f"Valor fora do intervalo 1-{N}")

        self.celulas[(i, j)] = val
        return self

    def matriz(self):
        N= self.n*self.n
        m = []
        for i in range(N):
            linha = []
            for j in range(N):
                valor = self.celulas.get((i, j))
                if valor is None:
                    linha.append(0)
                else:
                    linha.append(valor)
            m.append(linha)
        return m
```

R2- Cubo

Subclasse de Box que representa o bloco n*n cujo canto superior esquerdo é a célula (i*n ,j*n).

In [3]:
```
class cube(box):
    def __init__(self, n, i, j):
        box.__init__(self, n)
        for y in range(n):
            for x in range(n):
                self.add(i*n+y, j*n+x)
```

R3- Path

Subclasse de Box que representa um caminho reto, vertical ou horizontal.

Se um caminho nao for vertical ou horizontal levanta um erro do tipo ValueError

Vê a direção do caminho e vai adicionando as células passo a passo .

In [4]:
```
class path(box):
    def __init__(self, n, inicio, fim):
        box.__init__(self, n)
        inicio_y, inicio_x = inicio
        fim_y, fim_x = fim

        if inicio_y != fim_y and inicio_x != fim_x:
            raise ValueError("Tem que ser um troço reto")
        dir_y = 0
        if  fim_y > inicio_y: dir_y = 1
        if  fim_y < inicio_y: dir_y = -1

        dir_x = 0
        if  fim_x > inicio_x: dir_x = 1
        if  fim_x < inicio_x: dir_x = -1

        i, j = inicio_y, inicio_x
        self.add(i, j)
        while(i, j) != (fim_y, fim_x):
            i+= dir_y
            j+= dir_x
            self.add(i, j)
```

R4- Geração aleatória de pistas

O método pistas_aleatorias(n, k) retorna uma box com k células aleatórias preenchidas, k tem o valor padrão de n.

Se uma célula que já foi escolhida for escolhida de novo, escolhe-se outra célula aleatóriamente, tendo o resultado sempre k células diferentes.

In [5]:
```
def pistas_aleatorias(n, k = None):
    N = n*n
    if k is None:
        k =n
    b = box(n)
    while len(b.celulas) < k:
        i = randint(0, N-1)
        j = randint(0, N-1)
        if (i, j) not in b.celulas:
            b.add(i, j, randint(1, N))
    return b
```

R5- Modelo

Representa a grelha inteira

O construtor cria o modelo e uma variável inteira por célula.

O método add_grupos(self, grupos) recebe grupos, box, cube, path ou pistas aleatórias, e para cada um fixa as células que têm valor e certifica-se que são todas diferentes.

O método resolucao(self) corre o solver. Se terminar em OPTIMAL retorna a grelha preenchida. Se não houver solução retorna None.

In [6]:
```
class Modelo:
    def __init__(self, n):
        self.n = n
        N = self.n*self.n
        self.modelo = cp_model.CpModel()
        self.casas = []
        for i in range(N):
            linha = []
            for j in range(N):
                linha.append(self.modelo.NewIntVar(1, N,
"x"+str(i)+"_"+str(j)))
            self.casas.append(linha)

    def add_grupos(self, grupos):
        for grupo in grupos:
            for (i, j), valor in grupo.celulas.items():
                if valor is not None:
                    self.modelo.Add(self.casas[i][j] == valor)
            vars = []
            for (i, j) in grupo.celulas:
                vars.append(self.casas[i][j])
            self.modelo.AddAllDifferent(vars)

    def resolucao(self):
        N = self.n*self.n
        solver = cp_model.CpSolver()
        stat = solver.Solve(self.modelo)
        if stat != cp_model.OPTIMAL:
            return None
        grelha = []
        for i in range(N):
            linha = []
            for j in range(N):
                linha.append(solver.Value(self.casas[i][j]))
            grelha.append(linha)
        return grelha
```

R6 - Sudoku

Junta tudo para montar e resolver o Sudoku completo.

A função sudoku(n) gera as pistas com o método pistas_aleatorias, cria os grupos e mete-os no modelo com add_grupos. Depois resolve com o método resolucao.

Como as pistas são aleatórias podem tornar a solução impossível. Neste caso gera um novo modelo com novas pistas com um máximo de 100 tentativas. Se todas falharem, devolve (None, None).

Retorna a grelha resolvida e as pistas, para se poder verificar que foram respeitadas.

In [7]:

```
def sudoku(n):
    N = n*n
    for _ in range(100):
        pistas = pistas_aleatorias(n, None)
        grupos = []
        for i in range(N):
            grupos.append(path(n, (i, 0), (i, N-1)))
            grupos.append(path(n, (0, i), (N-1, i)))

        for i in range(n):
            for j in range(n):
                grupos.append(cube(n, i, j))

        grupos.append(pistas)

        modelo = Modelo(n)
        modelo.add_grupos(grupos)
        solucao = modelo.resolucao()
        if solucao is not None:
            return solucao, pistas
    return None, None
```

Testes

Recorremos ao ChatGPT para fazer os testes.

https://chatgpt.com/share/6ac55b07-7620-83eb-9d0c-068174d6cf98

In [8]:
```python
def teste_linhas_colunas_blocos(grelha, n):
    N = n * n
    esperado = set(range(1, N + 1))

    # Linhas
    for i in range(N):
        assert set(grelha[i]) == esperado
        assert len(grelha[i]) == len(set(grelha[i]))

    # Colunas
    for j in range(N):
        coluna = [grelha[i][j] for i in range(N)]
        assert set(coluna) == esperado
        assert len(coluna) == len(set(coluna))

    # Blocos n x n
    for bi in range(n):
        for bj in range(n):
            bloco = []

            for i in range(bi * n, bi * n + n):
                for j in range(bj * n, bj * n + n):
                    bloco.append(grelha[i][j])

            assert set(bloco) == esperado
            assert len(bloco) == len(set(bloco))

    print("✓ Teste 1 passou")
```

In [9]:
```python
def teste_pistas(solucao, pistas):
    for (i, j), valor in pistas.celulas.items():
        assert solucao[i][j] == valor

    print("✓ Teste 2 passou")
```

In [10]:
```python
def teste_add():
    b = box(3)

    # Coordenadas inválidas
    try:
        b.add(-1, 0, 1)
        assert False
    except ValueError:
        pass

    try:
        b.add(9, 0, 1)
        assert False
    except ValueError:
        pass
```

```
    try:
        b.add(0, -1, 1)
        assert False
    except ValueError:
        pass

    try:
        b.add(0, 9, 1)
        assert False
    except ValueError:
        pass

    # Valores inválidos
    try:
        b.add(0, 0, 0)
        assert False
    except ValueError:
        pass

    try:
        b.add(0, 0, 10)
        assert False
    except ValueError:
        pass

    print("✓ Teste 3 passou")
```

In [17]:

```
solucao, pistas = sudoku(3)

teste_linhas_colunas_blocos(solucao, 3)
teste_pistas(solucao, pistas)
teste_add()

print("✓ Todos os testes 9x9 passaram")


solucao, pistas = sudoku(2)

teste_linhas_colunas_blocos(solucao, 2)
teste_pistas(solucao, pistas)
teste_add()

print("✓ Todos os testes 4x4 passaram")
```

```
✓ Teste 1 passou
✓ Teste 2 passou
✓ Teste 3 passou
✓ Todos os testes 9x9 passaram
✓ Teste 1 passou
✓ Teste 2 passou
✓ Teste 3 passou
✓ Todos os testes 4x4 passaram
```

Extenção opcional- X-Sudoku

Acrescenta um grupo box para cada uma das duas diagonais principais, também elas restritas a "todos diferentes".

A diagonal descendente é a que tem início nas coordenadas (0, 0) e fim nas coordenadas (N-1, N-1).

A diagonal ascendente é a que tem início nas coordenadas (0,N-1) e fim nas coordenadas (N-1, 0).

In [12]:
```
def diagonais(n):
    N = n*n
    desc = box(n)
    asc = box(n)
    for i in range(N):
        desc.add(i, i)
        asc.add(i, N-i-1)
    return desc, asc

def sudoku_x(n):
    N = n*n
    for _ in range(100):
        pistas = pistas_aleatorias(n, None)
        grupos = []
        for i in range(N):
            grupos.append(path(n, (i, 0), (i, N-1)))
            grupos.append(path(n, (0, i), (N-1, i)))
        for i in range(n):
            for j in range(n):
                grupos.append(cube(n, i, j))
        grupos = grupos + list(diagonais(n))
        grupos.append(pistas)

        modelo = Modelo(n)
        modelo.add_grupos(grupos)
        solucao = modelo.resolucao()
        if solucao is not None:
            return solucao, pistas
    return None, None
```

In [13]:
```python
def teste_diagonais(grelha, n):
    N = n * n
    esperado = set(range(1, N + 1))

    desc = [grelha[i][i] for i in range(N)]
    assert set(desc) == esperado
    assert len(desc) == len(set(desc))

    asc = [grelha[i][N - 1 - i] for i in range(N)]
    assert set(asc) == esperado
    assert len(asc) == len(set(asc))

    print("Teste das diagonais passou")
```

In [16]:
```python
solucao, pistas = sudoku_x(3)

teste_linhas_colunas_blocos(solucao, 3)
teste_pistas(solucao, pistas)
teste_add()
teste_diagonais(solucao, 3)

print("✓ Todos os testes 9x9 passaram")

solucao, pistas = sudoku_x(2)

teste_linhas_colunas_blocos(solucao, 2)
teste_pistas(solucao, pistas)
teste_add()
teste_diagonais(solucao, 2)

print("✓ Todos os testes 4x4 passaram")
```

```
✓ Teste 1 passou
✓ Teste 2 passou
✓ Teste 3 passou
Teste das diagonais passou
✓ Todos os testes 9x9 passaram
✓ Teste 1 passou
✓ Teste 2 passou
✓ Teste 3 passou
Teste das diagonais passou
✓ Todos os testes 4x4 passaram
```